ELE 888 Guide

Project walkthrough: what each piece does, and how to run it start to finish

What this project is

This repository re-implements the core machine-learning algorithms from my ELE 888 (Intelligent Systems) labs **in Python, from the algorithm up** — and then checks each one against scikit-learn so there's proof the hand-written version actually works.

The original labs were in MATLAB with much of the structure handed to us. The value of redoing them this way is that it shows I can *build* k-means, backpropagation, an SVM and PCA — not just call a library — which is what an ML/AI role actually wants to see.

Four topics, matching the labs:

1. **Clustering** — k-means, DBSCAN, k-nearest-neighbours (Lab 3)
2. **Neural networks** — forward prop, backprop, stochastic gradient descent (Lab 3)
3. **Support vector machines** — the large-margin classifier (Lab 3)
4. **PCA & image compression** — dimensionality reduction (Lab 4)

There are **no unit tests** by design — the "test" is that every algorithm reproduces scikit-learn's answer, which the scripts print on every run.

The shape of the repository

```
ele888-ml-labs/
├── README.md               the results table and the story
├── requirements.txt        numpy, pandas, scikit-learn, matplotlib
├── pyproject.toml          lets you `pip install -e .` and import ele888 anywhere
├── Makefile                make setup / run / clustering / neuralnet / svm / images /
pca
├── run_all.py              runs all 7 experiments in order
├── data/
│   └── heart.csv           South African heart-disease dataset (Lab 3)
├── src/ele888/             THE LIBRARY — the from-scratch algorithms
│   ├── distances.py
│   ├── clustering/         kmeans.py, dbscan.py, knn.py
│   ├── neural_net/         activations.py, network.py
│   ├── svm/                linear_svm.py
│   └── decomposition/      pca.py
├── scripts/                THE EXPERIMENTS — 7 runnable demos
└── figures/                plots, generated by `make run`
```

The split is deliberate: `src/ele888/` is reusable library code (each class uses the same `fit` / `predict` names scikit-learn does), and `scripts/` are the experiments that import that library and produce results.

The library, file by file

`distances.py`

One place for Euclidean distance, because k-means, k-NN **and** DBSCAN all ask the same "how far apart are two points?" question. `euclidean_matrix` is the vectorised version (distance from every point to every centre at once) — that's what keeps k-means fast enough to re-run ten times.

`clustering/kmeans.py`

The full k-means loop: pick `k` centres, assign each point to its nearest centre, move each centre to the mean of its points, repeat until the assignments stop changing. On top of that it does Lab 4's **restart trick** — run the whole thing `n_init` times and keep the run with the lowest *dispersion cost* (total squared distance to centres), because a bad random start gives a bad clustering.

`clustering/dbscan.py`

Density-based clustering. Unlike k-means you don't tell it how many clusters there are — you give it `eps` (how close counts as neighbours) and `min_points` (how many neighbours make a dense region). It grows clusters out from dense points and labels the rest noise (`-1`). Good for oddly-shaped clusters and spotting outliers.

`clustering/knn.py`

k-nearest-neighbours **classifier**. It doesn't train — it just remembers the data, and to classify a new point it finds the `k` closest labelled points and takes a majority vote. All the cost is at prediction time, which is exactly why it gets slow on big data.

`neural_net/activations.py`

`sigmoid`, `tanh`, `relu` and their derivatives. Each activation needs its derivative because backprop uses `g'(z)` to push the error back through a layer.

`neural_net/network.py`

The neural network: forward propagation, backpropagation, and stochastic gradient descent, kept close to the lab's notation (`s = Wᵀx`, `x = g(s)`, output sensitivity `δ = (x − y)·g'(s)`). It handles both **binary** (one output neuron, threshold at 0.5) and **multi-class** (one neuron per class, pick the highest) classification, with optional L2 regularisation.

`svm/linear_svm.py`

A soft-margin linear SVM trained by sub-gradient descent on the **hinge loss**. This makes the `C` dial concrete (large `C` = punish every mistake, narrow margin; small `C` = wider margin). For curved boundaries you use a kernel — which is where I switch to scikit-learn's `SVC`, because the kernel trick is where hand-rolling stops teaching you much.

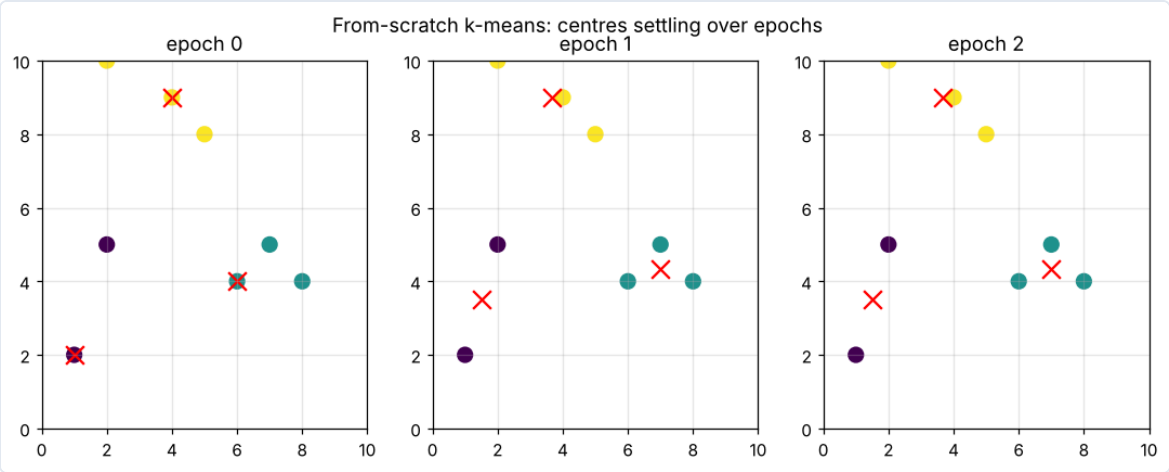
`decomposition/pca.py`

PCA by SVD of the centred data: `fit` finds the top principal components, `transform` projects data down to `k` dimensions, and `inverse_transform` rebuilds an approximation — which is the image-compression step from Lab 4.

The experiments (what you'll see when you run them)

1 — k-means on the toy set

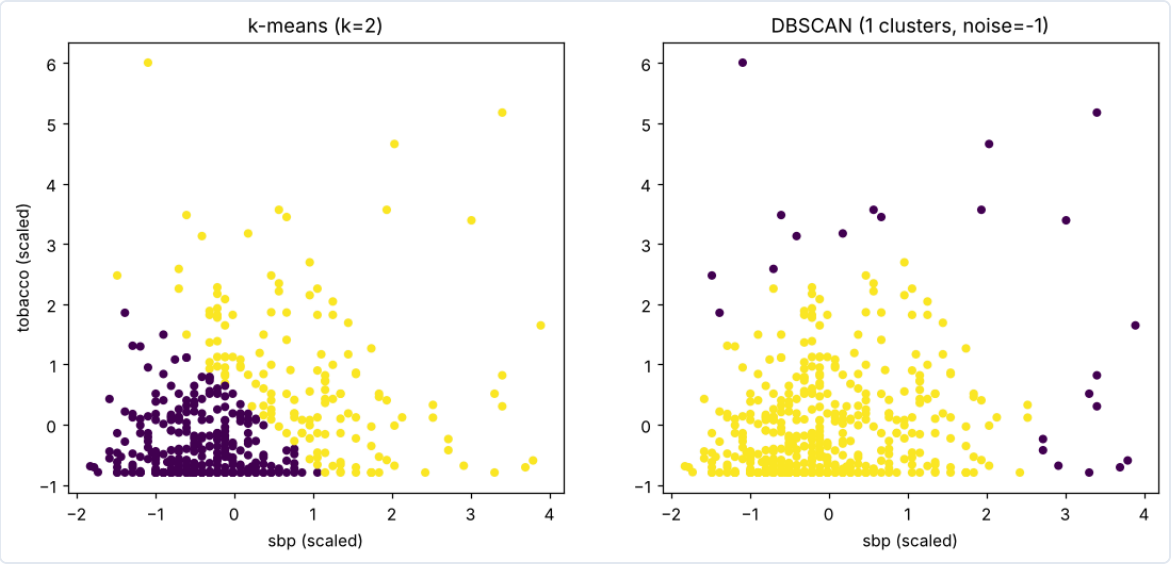
The eight points from Lab 3, clustered into three groups, with the centres shown moving each epoch. The from-scratch version lands on the **exact** dispersion cost scikit-learn gets (14.33).



k-means centres settling over epochs

2 — clustering & k-NN on real heart data

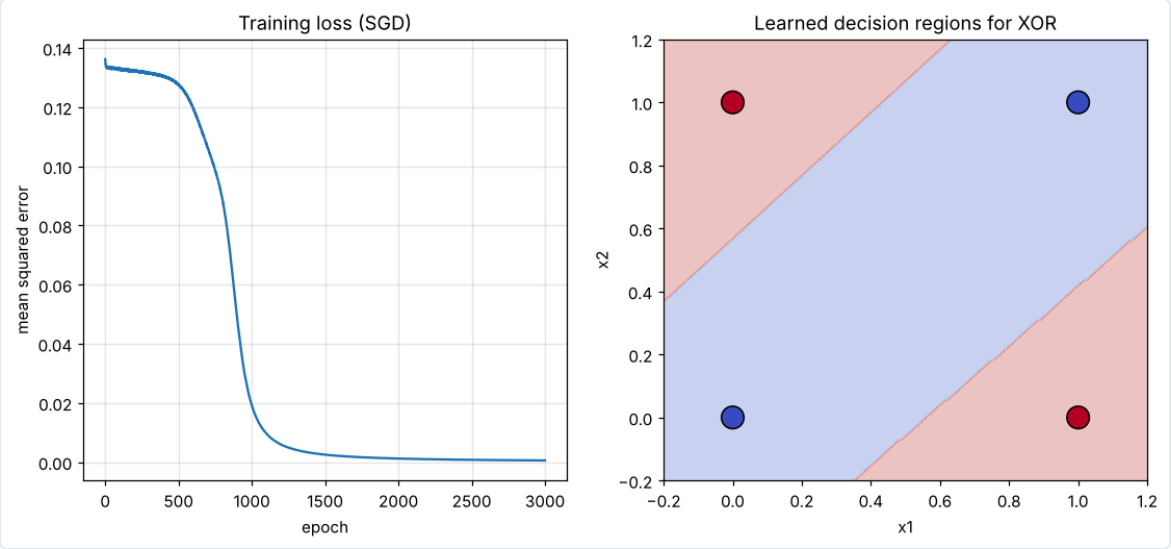
k-means and DBSCAN on the heart-disease dataset, plus a k-NN classifier predicting heart disease (`chd`). The from-scratch k-NN matches scikit-learn's accuracy at **every** value of k. The accuracy is modest (~63%) — which is the honest result for predicting heart disease from a couple of features, not a bug.



k-means vs DBSCAN on the heart data

3 — the XOR problem

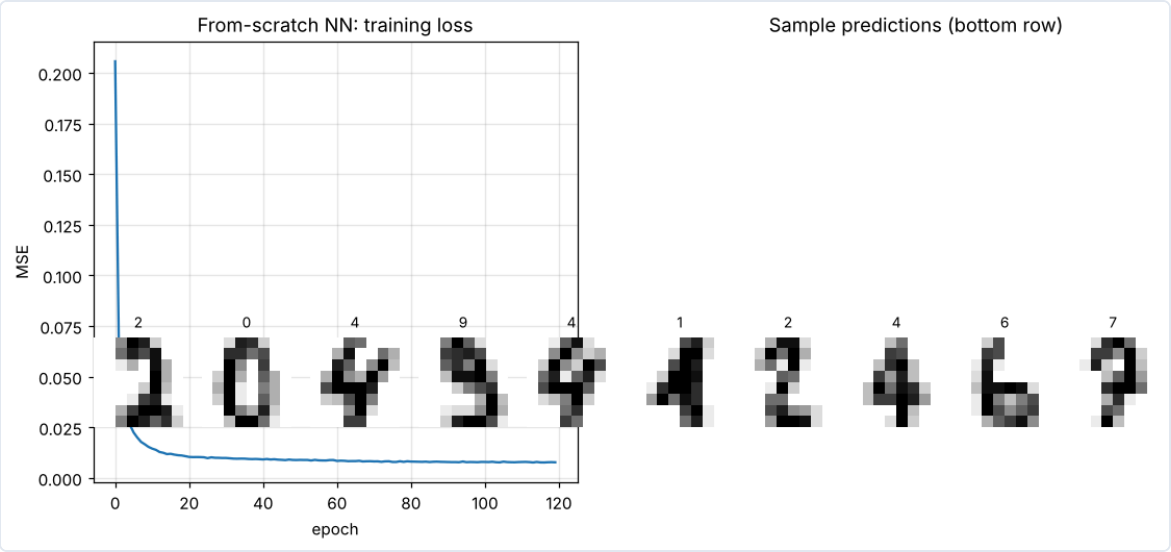
XOR is the classic example of data that **no straight line can separate**. A network with one hidden layer bends the boundary and solves it — 100% accuracy — and you can watch the loss fall and see the decision regions it learns.



XOR: training loss and learned decision regions

4 — handwritten digit recognition

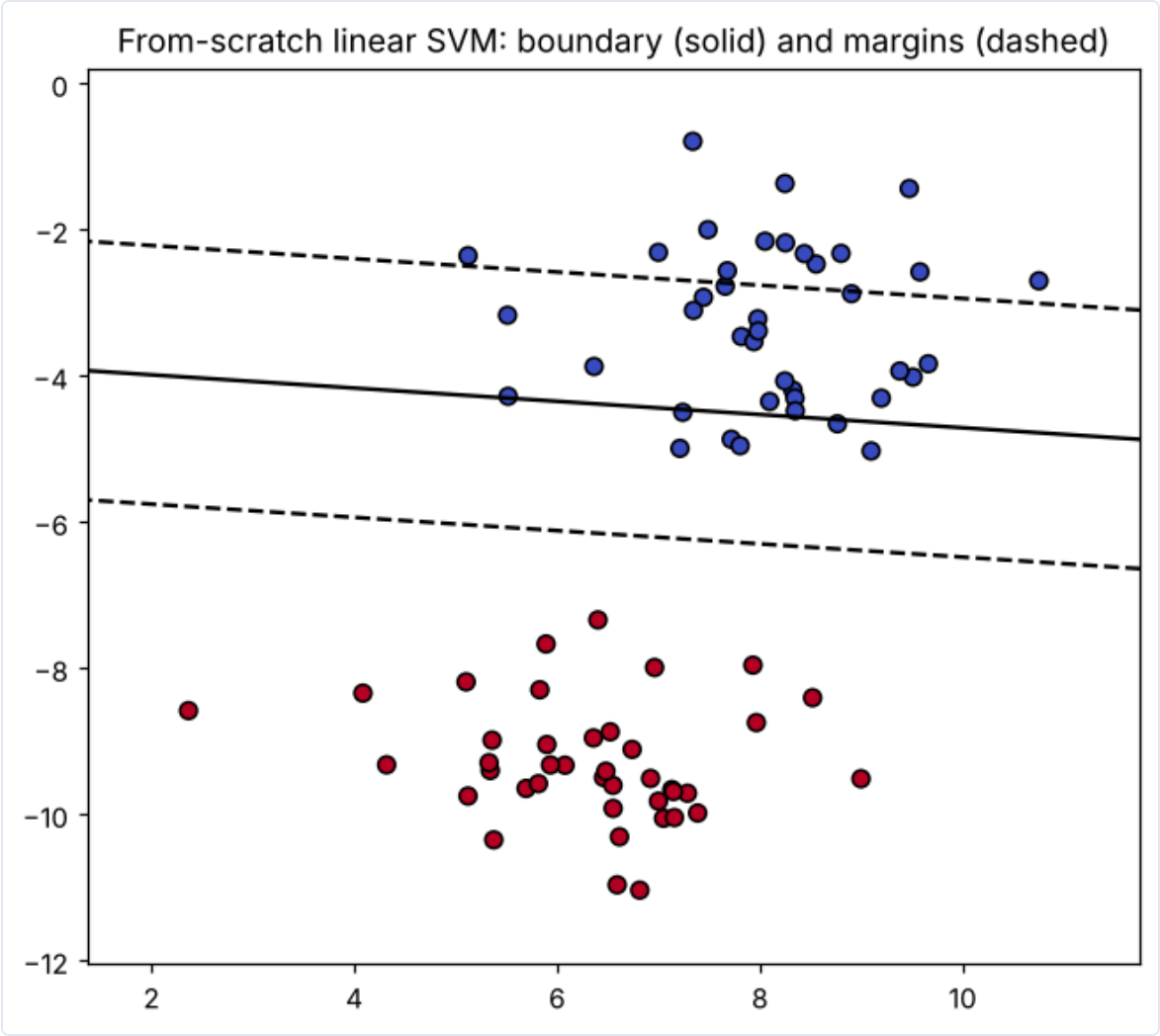
The from-scratch neural network on 10-class digits (64 features each). It reaches **98%**, the same score scikit-learn's `MLPClassifier` gets — and that's the same kind of model a Keras `Dense` network would be.



From-scratch network: loss curve and sample predictions

5 — SVM vs neural network

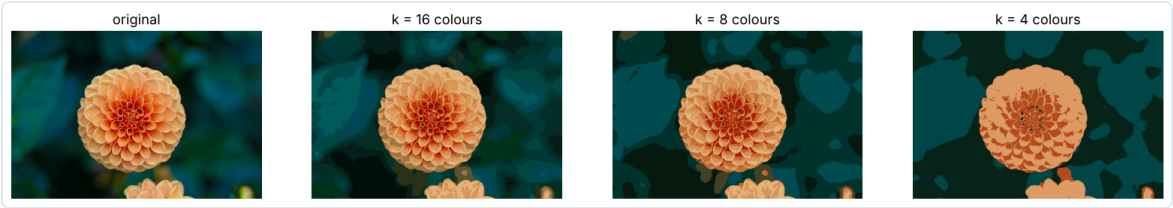
The large-margin boundary the from-scratch SVM finds (solid line, dashed margins), and then telling a `3` from an `8` four ways: from-scratch SVM (97%), scikit-learn linear SVC (99%), **RBF-kernel SVC (100%)**, and the neural net (99%). The kernel winning is the whole argument for kernels, in one number.



From-scratch linear SVM: boundary and margins

6 — image colour quantization with k-means

Every pixel is a point in (R, G, B) colour space. Run k-means and repaint each pixel as its nearest centre, and you get the same image drawn with only 16 / 8 / 4 colours — image compression by clustering, straight from Lab 4.

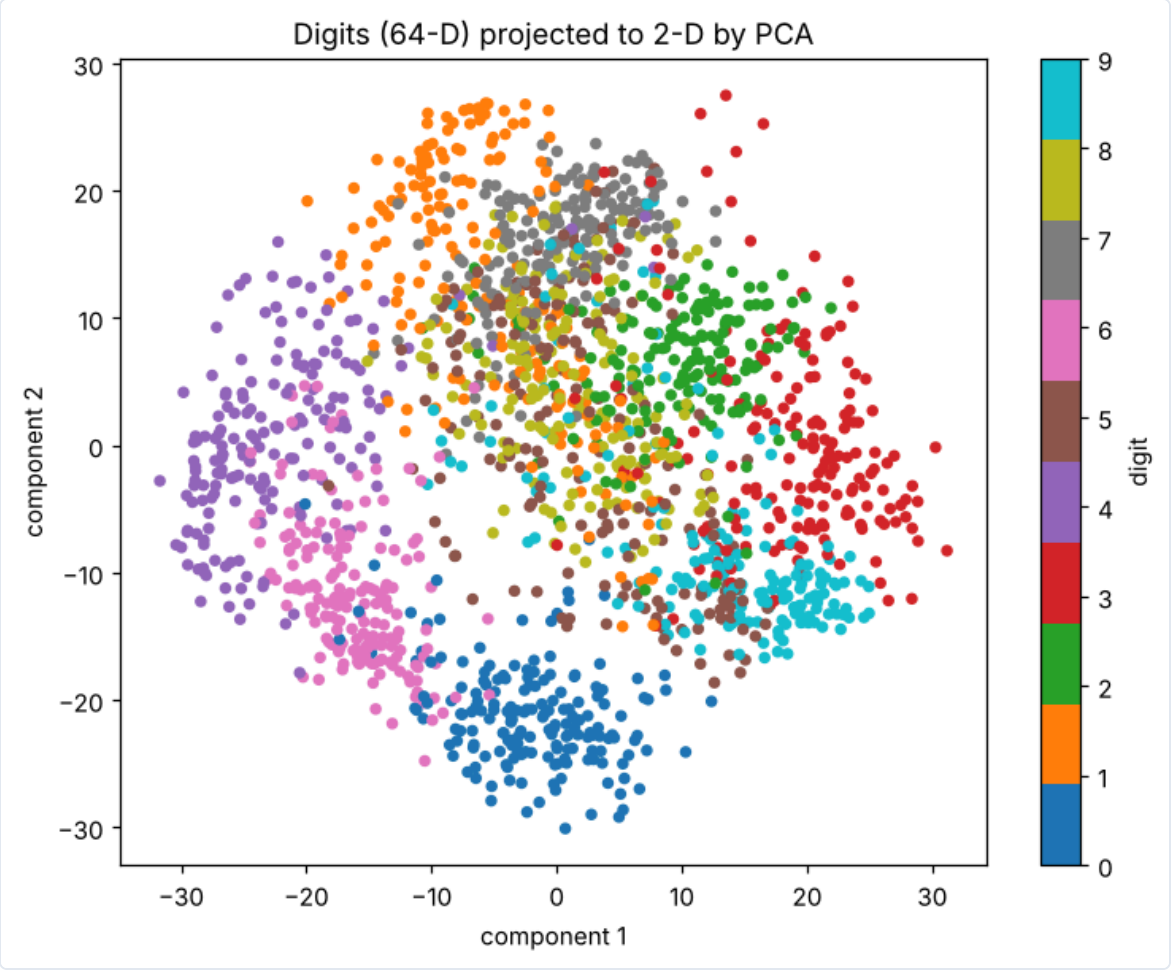


An image reduced to 16, 8 and 4 colours

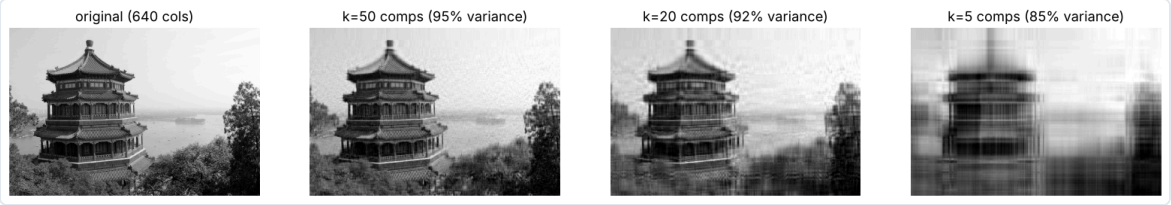
7 — PCA for visualization and compression

Two uses of PCA: squashing 64-dimensional digits to 2-D so all ten classes fit on one plot, and compressing an image by keeping only its top components — 50 components hold 95% of the

variance and the image is still clearly readable.



Digits projected to 2-D by PCA



PCA image compression at 50, 20 and 5 components

Running it, start to finish

1. Install Python 3.10+ and check it:

```
python3 --version
```

2. Install the dependencies (optionally in a virtual environment first):

```
        python3 -m venv .venv && source .venv/bin/activate     # optional but tidy
        make setup                                              # pip install -r
requirements.txt
```

3. Run everything (~20 seconds):

```
        make run
```

You'll see each experiment print its results, and the plots appear in `figures/`. To run one group: `make clustering`, `make neuralnet`, `make svm`, `make images`, or `make pca`.

4. Use the algorithms in your own code:

```
        import sys; sys.path.insert(0, "src")     # or run `pip install -e .` once
        from ele888.clustering.kmeans import KMeans
        labels = KMeans(n_clusters=3).fit_predict(X)
```

Putting it on GitHub

Create an empty repo called `ele888-ml-labs` on GitHub (no README, so there's nothing to merge), then:

```
        cd ele888-ml-labs
        git init
        git add .
        git commit -m "ELE 888: machine-learning algorithms from scratch"
        git branch -M main
        git remote add origin https://github.com/munevvercoskun/ele888-ml-labs.git
        git push -u origin main
```

The `figures/` PNGs are git-ignored because `make run` regenerates them. If you'd rather show them on the repo page, delete the last line of `.gitignore` and commit them.

What to say about it

If someone asks about this project, the honest, strong version is: *"I re-implemented the ELE 888 lab algorithms — k-means, backpropagation, an SVM, PCA — from scratch in Python, and verified each one against scikit-learn. My from-scratch neural network gets the same 98% on digit recognition that scikit-learn's MLP does, which told me I'd implemented backprop correctly and not just something that happens to run."*

That's the difference between "I've used scikit-learn" and "I understand what scikit-learn is doing" — and this repo is the proof of the second one.